แผนการเรียนรายวิชาวิทยาการคำนวณ/คอมพิวเตอร์

ระดับชั้น: ประถมศึกษา
ระยะเวลา: 3 ชั่วโมง
วิชา: วิทยาการคำนวณ/คอมพิวเตอร์
หน่วยการเรียนรู้ : การเขียนโปรแกรม Pythonเบื้องต้น

มาตรฐานการเรียนรู้และตัวชี้วัด

ว 4.1 เข้าใจและใช้เทคโนโลยีสารสนเทศและการสื่อสารในการสืบค้นข้อมูล การเรียนรู้ การสื่อสาร การแก้ปัญหา

ว 7.1 เข้าใจกระบวนการออกแบบเทคโนโลยี สามารถออกแบบและสร้างชิ้นงาน ประดิษฐ์คิดค้น เพื่อการดำรงชีวิต

ว 1.1 เข้าใจกระบวนการทางวิทยาศาสตร์และสามารถนำไปใช้ในการแก้ปัญหา

ง 2.1 มีทักษะการใช้เทคโนโลยี การคิดเชิงออกแบบ และกระบวนการทำงาน

ง 4.1 เข้าใจ เห็นคุณค่า และใช้กระบวนการเทคโนโลยีสารสนเทศในการสืบค้นข้อมูล การเรียนรู้ การสื่อสาร การแก้ปัญหา และการทำงานอาชีพอย่างมีประสิทธิภาพ ประสิทธิผล และมีคุณธรรม

ตัวชี้วัด

ด้านความรู้ (Knowledge - K)

K1 อธิบายความหมาย ประโยชน์ และความเป็นมาของภาษา Python ได้
K2 ระบุส่วนประกอบหลักของโปรแกรม Python (Editor, Console, Menu) ได้
K3 บอกข้อดีของภาษา Python เปรียบเทียบกับภาษาอื่นๆ ได้
K4 อธิบายความแตกต่างระหว่าง Scratch และ Python ได้
K5 อธิบายการไหลของโปรแกรม (Program Flow) จากบนลงล่างได้
K6 บอกกฎการตั้งชื่อตัวแปรและ Reserved Words ที่ห้ามใช้ได้
K7 จำแนกชนิดข้อมูล (int, float, str, bool) ได้ถูกต้อง
K8 อธิบายการใช้งานคำสั่ง print(), input() และตัวดำเนินการทางคณิตศาสตร์ได้

ด้านทักษะ (Skills - S)

S1 เปิดและใช้งานโปรแกรม Python (IDLE/Replit) ได้
S2 ประกาศและใช้งานตัวแปรได้อย่างถูกต้องตามกฎการตั้งชื่อ
S3 ใช้คำสั่ง print() แสดงผลข้อมูลได้หลากหลายรูปแบบ
S4 ใช้คำสั่ง input() รับข้อมูลจากผู้ใช้ได้
S5 แปลงชนิดข้อมูล (type conversion) ด้วย int(), float(), str() ได้
S6 เขียนโค้ดคำนวณทางคณิตศาสตร์ (+, -, *, /, **) ได้
S7 ตรวจสอบและแก้ไขข้อผิดพลาดเบื้องต้นในโค้ดได้
S8 เขียนโปรแกรม Python อย่างง่ายเพื่อแก้ปัญหาในชีวิตประจำวันได้

ด้านคุณลักษณะ (Attributes - A)

A1 ความละเอียดรอบคอบและความพิถีพิถันในการเขียนโค้ด
A2 ความคิดสร้างสรรค์และทักษะการแก้ปัญหาอย่างเป็นระบบ
A3 ความอดทน มุ่งมั่น และไม่ท้อแท้เมื่อพบข้อผิดพลาด
A4 ความรับผิดชอบในการทำงานที่ได้รับมอบหมาย
A5 ความกระตือรือร้นและความสนใจในการเรียนรู้เทคโนโลยีใหม่ๆ
A6 จิตสำนึกในการใช้เทคโนโลยีอย่างสร้างสรรค์และมีจริยธรรม

วัตถุประสงค์การเรียนรู้
เมื่อจบบทเรียน นักเรียนสามารถ:
ด้านความรู้ (Cognitive Domain)

1. เข้าใจหลักการทำงานและวัตถุประสงค์ของภาษา Python
2. จำแนกส่วนประกอบของโปรแกรม Python ได้
3. เปรียบเทียบข้อดีข้อเสียของ Python กับภาษาโปรแกรมอื่นๆ
4. เข้าใจการไหลของโปรแกรม (Program Flow)

ด้านทักษะ (Psychomotor Domain)

5. สามารถติดตั้งและเปิดใช้งานโปรแกรม Python ได้
6. สามารถประกาศและใช้งานตัวแปรได้อย่างถูกต้อง
7. สามารถเขียนโค้ด Python เบื้องต้นได้

ด้านคุณลักษณะ (Affective Domain)

8. มีความละเอียดรอบคอบในการเขียนโปรแกรม
9. มีความคิดสร้างสรรค์และทักษะการแก้ปัญหา
10. มีความอดทนและมุ่งมั่นในการเรียนรู้

คาบเรียนที่ 11: ความรู้เบื้องต้นและการติดตั้ง(60 นาที)

1. รู้จักกับภาษาคอมพิวเตอร์

1.1. ภาษาคอมพิวเตอร์มีหลายระดับ ตั้งแต่ระดับต่ำใกล้เครื่องอย่าง **ภาษาเครื่อง (Machine Language)** และ **ภาษาแอสเซมบลี (Assembly)** ไปจนถึงภาษาระดับสูงที่มนุษย์เข้าใจง่ายกว่า เช่น **Python, Java, C++, JavaScript, C#** ซึ่งมีการใช้งานหลากหลาย ตั้งแต่พัฒนาเว็บ แอปพลิเคชัน ไปจนถึงงานวิทยาศาสตร์และวิศวกรรมศาสตร์

1.1.1. ภาษาระดับต่ำ (Low-Level Languages)
ภาษาเครื่อง (Machine Language): ภาษาที่คอมพิวเตอร์เข้าใจโดยตรง ประกอบด้วยเลข 0 และ 1 เป็นภาษาพื้นฐานที่สุด แต่ยากสำหรับมนุษย์.
ภาษาแอสเซมบลี (Assembly Language): ใช้สัญลักษณ์ย่อ (Mnemonics) แทนคำสั่งเลขฐานสอง ทำให้เข้าใจง่ายกว่าภาษาเครื่อง แต่ยังคงใกล้เคียงฮาร์ดแวร์.

1.1.2. **ภาษาระดับสูง (High-Level Languages)** ภาษานี้ใกล้เคียงภาษามนุษย์ ทำให้เขียนง่าย และต้องมีโปรแกรมตัวแปล (Compiler/Interpreter) แปลเป็นภาษาเครื่อง.
Python: ใช้งานหลากหลาย ตั้งแต่ AI, วิทยาศาสตร์ข้อมูล, เว็บไซต์, เรียนรู้ง่าย.
Java: ใช้พัฒนาแอปฯ องค์กร, Android, และระบบที่ต้องการความเสถียร . **C++/C:** เน้นประสิทธิภาพสูง เหมาะกับเกม, ระบบฝังตัว (Embedded Systems).
JavaScript: สำคัญมากสำหรับพัฒนาเว็บ (Client-side).
C# (ซีชาร์ป): พัฒนาโดย Microsoft, ใช้กับ Windows, เกม (Unity).
PHP: นิยมใช้พัฒนาเว็บฝั่งเซิร์ฟเวอร์.
Ruby: เน้นพัฒนาอย่างรวดเร็วด้วยเฟรมเวิร์ก Ruby on Rails.

1.1.3. **ภาษาเฉพาะทาง**
SQL: ภาษาสำหรับจัดการฐานข้อมูล.
HTML/CSS: ใช้สร้างโครงสร้างและจัดรูปแบบหน้าเว็บ (ไม่ใช่ภาษาโปรแกรมมิ่งโดยตรง แต่สำคัญในการพัฒนาเว็บ).

2. รู้จักกับ Python

2.1. Python คืออะไร และเราเรียนเพื่ออะไร

Python (ออกเสียงว่า "ไพธอน") เป็นภาษาโปรแกรมที่ใช้สั่งงานคอมพิวเตอร์ คิดง่ายๆ ว่า เหมือนกับภาษาอังกฤษ ฝรั่งเศส ที่เราใช้คุยกับคน แต่ Python คือภาษาที่เราใช้คุยกับคอมพิวเตอร์! สร้างโดย Guido van Rossum คนเนเธอร์แลนด์ เมื่อปี 1991 ชื่อ Python มาจากรายการตลก "Monty Python's Flying Circus"

Python คือหนึ่งในภาษาโปรแกรมระดับสูงที่ใช้กันอย่างแพร่หลาย ทั้งในการสร้างเว็บไซต์ พัฒนาซอฟต์แวร์ รวมถึงงานวิเคราะห์ข้อมูล โดยถูกออกแบบเพื่อให้มีโครงสร้างและ ไวยากรณ์ของภาษาที่ไม่ซับซ้อน เข้าใจง่าย มีการใช้พัฒนาแอปพลิเคชันเว็บไซต์ รวมถึงแอปบนมือถือหรือ อุปกรณ์เคลื่อนที่ด้วย หน้าที่ของ Python ก็คือการทำงานแปลชุดคำสั่งทีละบรรทัดเพื่อป้อนเข้าสู่หน่วยประมวลผล ให้คอมพิวเตอร์ทำงานตามที่เราต้องการ หรือเรียกว่าการทำงานแบบ Interpreter นั่นเอง ด้วยภาษาที่ง่ายในการเขียน “Python” จึงมีความเหมาะสมสำหรับผู้ที่เริ่มต้นเขียนโปรแกรมไปจนถึงนักพัฒนาในองค์กรบริษัทใหญ่

ทำไมต้องเรียน Python?
เหตุผลที่ 1: Python อยู่รอบตัวเรา

- Instagram, YouTube, Spotify ที่เราใช้ทุกวัน → สร้างด้วย Python!
- Netflix แนะนำหนังที่เราชอบ → ใช้ Python!
- Google ค้นหาข้อมูล → มี Python ช่วย!

เหตุผลที่ 2: สร้างสิ่งเจ๋งๆ ได้

- เขียนเกมของตัวเอง
- สร้างแอพมือถือ

- ทำเว็บไซต์
- สั่งงานหุ่นยนต์

เหตุผลที่ 3: ง่ายที่สุดสำหรับมือใหม่

- อ่านเหมือนภาษาอังกฤษ
- เขียนสั้นกว่าภาษาอื่น
- มีคนช่วยเหลือเยอะมาก

ตัวอย่างเช่น

Java	Python

public class HelloWorld {
 	
public static void main(String[] args) {
 	
System.out.println("Hello");
 	
}
 	
}
	print("Hello")

ความต้องการในตลาดแรงงานสูง

ในยุคที่เทคโนโลยีเข้ามาเปลี่ยนแปลงแบบนี้ สกิลที่จะเข้าไปเสริมความแข็งแกร่งและเป็นที่ต้องการในตลาดแรงงานได้อย่างง่ายดายนั่นคือ
การเขียนโปรแกรม ก้าวแรกของการเรียน Python ถือเป็นการอัปเกรดสกิลในตัวน้องๆได้ดีเลยทีเดียว ไม่ว่าจะเป็นงานในสาย Data science และ Machine learning รวมไปถึงองค์กรขนาดใหญ่ระดับโลกอย่าง Netflix, Uber, Instagram และ Spotify ก็ยังนำภาษา Python ไปใช้สร้างแอพลิเคชันของตัวเองด้วย และด้วยความที่ Python เป็นภาษาที่สามารถใช้ได้หลายอย่าง จึงง่ายในการปรับและ นำไปใช้จริงในบริษัทต่างๆ

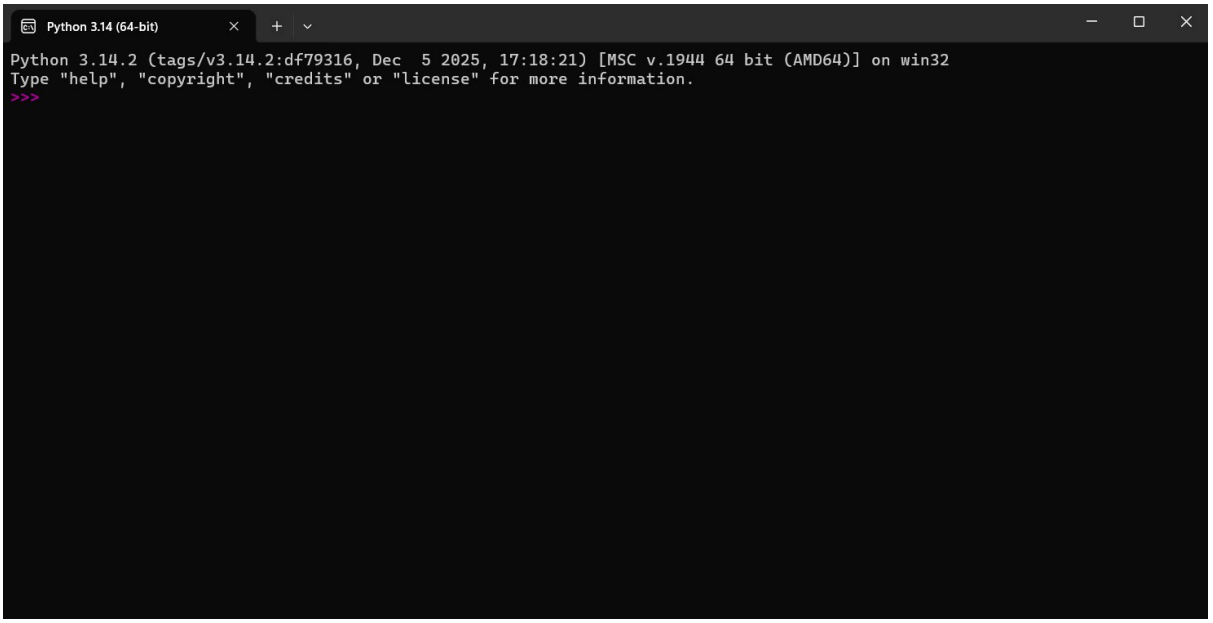
2.2. ส่วนประกอบหลักและหน้าตาของโปรแกรม Python

รู้จักกับ IDLE (โปรแกรมเขียน Python)
เวลาเราเปิด Python จะเจอ 2 หน้าต่างสำคัญ:

หน้าต่างที่ 1: Shell



หน้าต่างที่ 2: Editor (หน้าจอเขียนโค้ด)



2.3. โปรแกรมที่ใช้เขียนภาษา Python

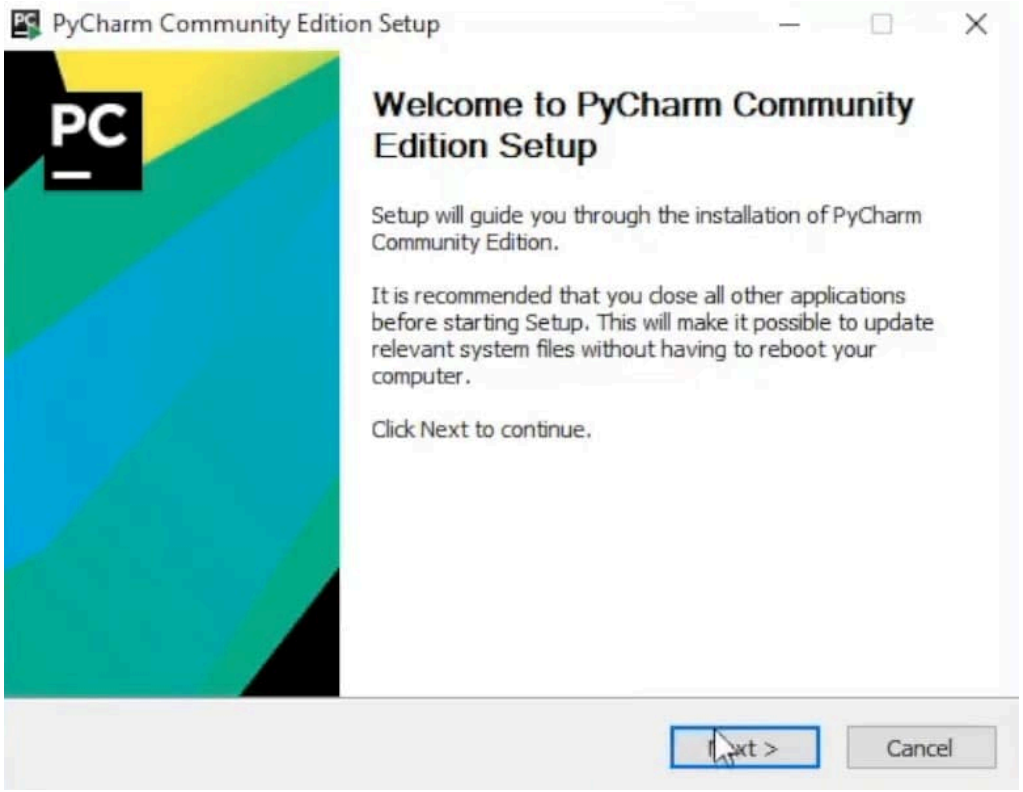
2.3.1. Pycharm

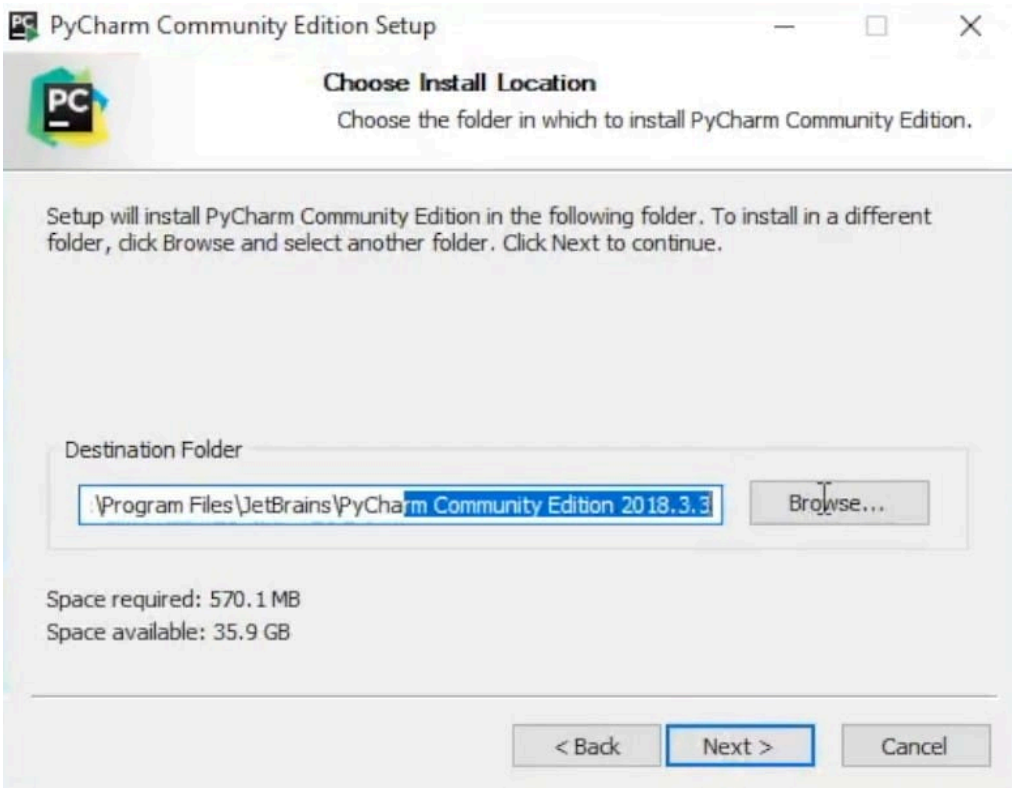
คือ Integrated Development Environment (IDE) หรือเครื่องมือพัฒนาแบบครบวงจรสำหรับภาษาโปรแกรม Python ที่ช่วยให้นักพัฒนาเขียนโค้ด, ดีบั๊ก, ทดสอบ และจัดการโปรเจกต์ได้ง่ายขึ้น มีฟีเจอร์อัจฉริยะ เช่น การเติมโค้ดอัตโนมัติ, วิเคราะห์โค้ด, รองรับเฟรมเวิร์กเว็บยอดนิยม (Django, Flask) และฐานข้อมูล, สามารถใช้งานได้บนหลายระบบปฏิบัติการ (Windows, macOS, Linux)

ดาวน์โหลด & ติดตั้ง PyCharm

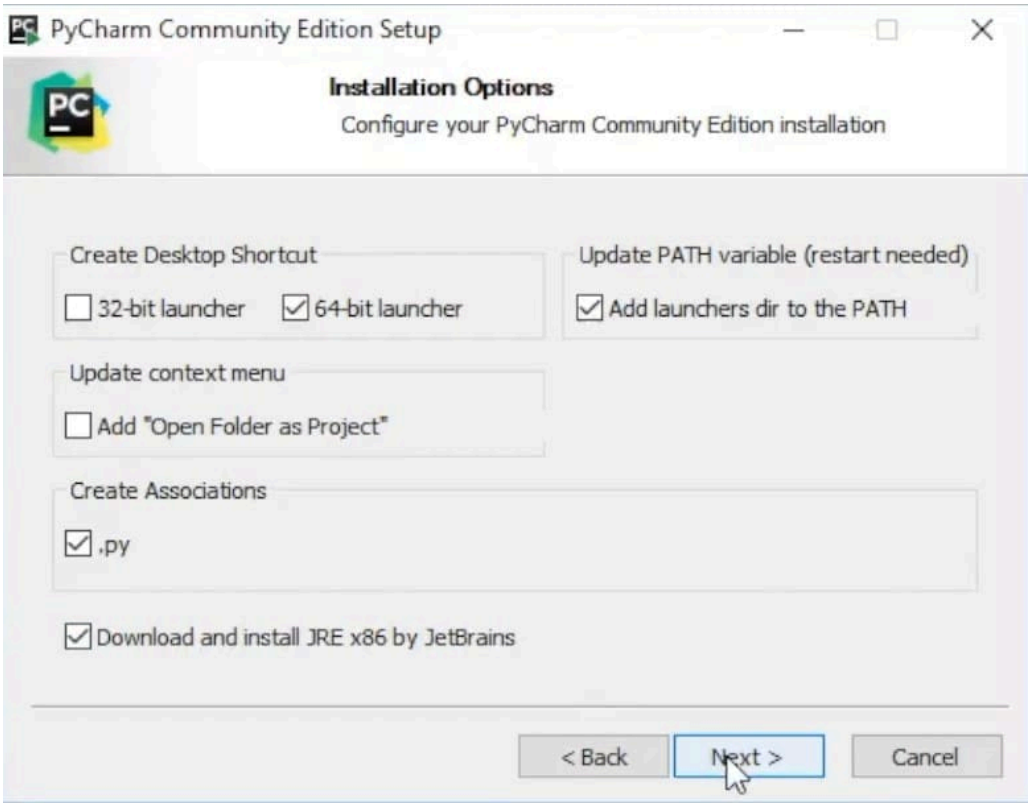
- เข้าไปที่เว็บ https://www.jetbrains.com/pycharm/download เลือกดาวน์โหลดเวอร์ชัน community

- เปิดไฟล์ที่ดาวน์โหลดขึ้นมา กด Next

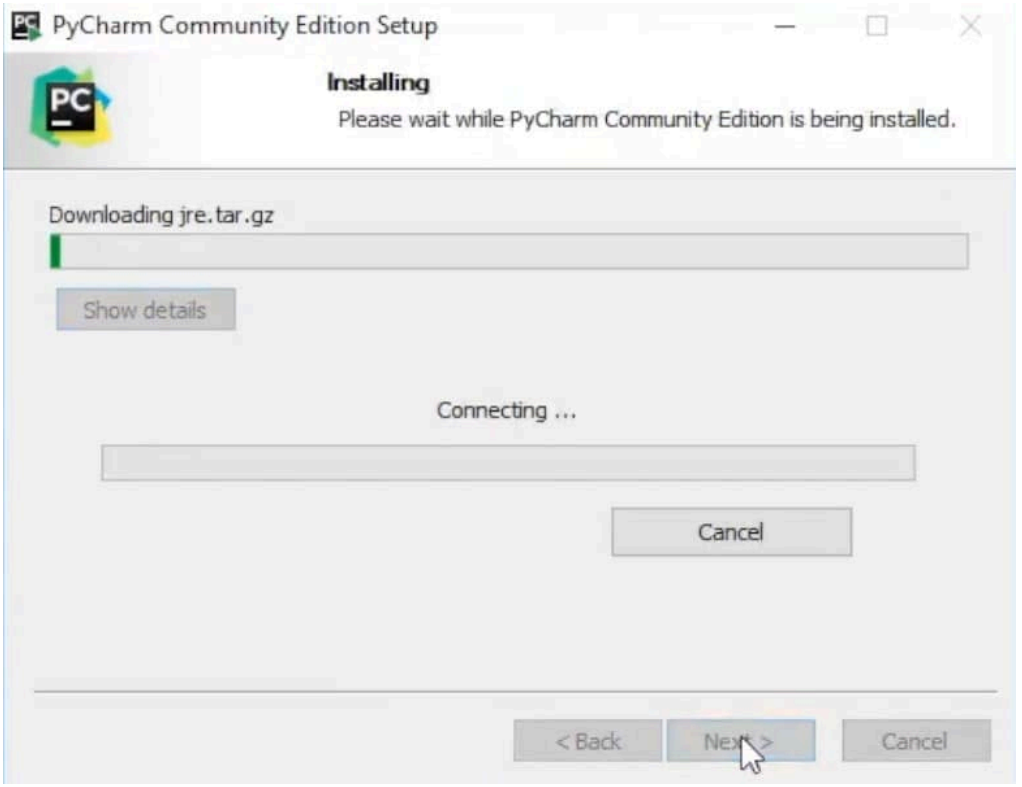




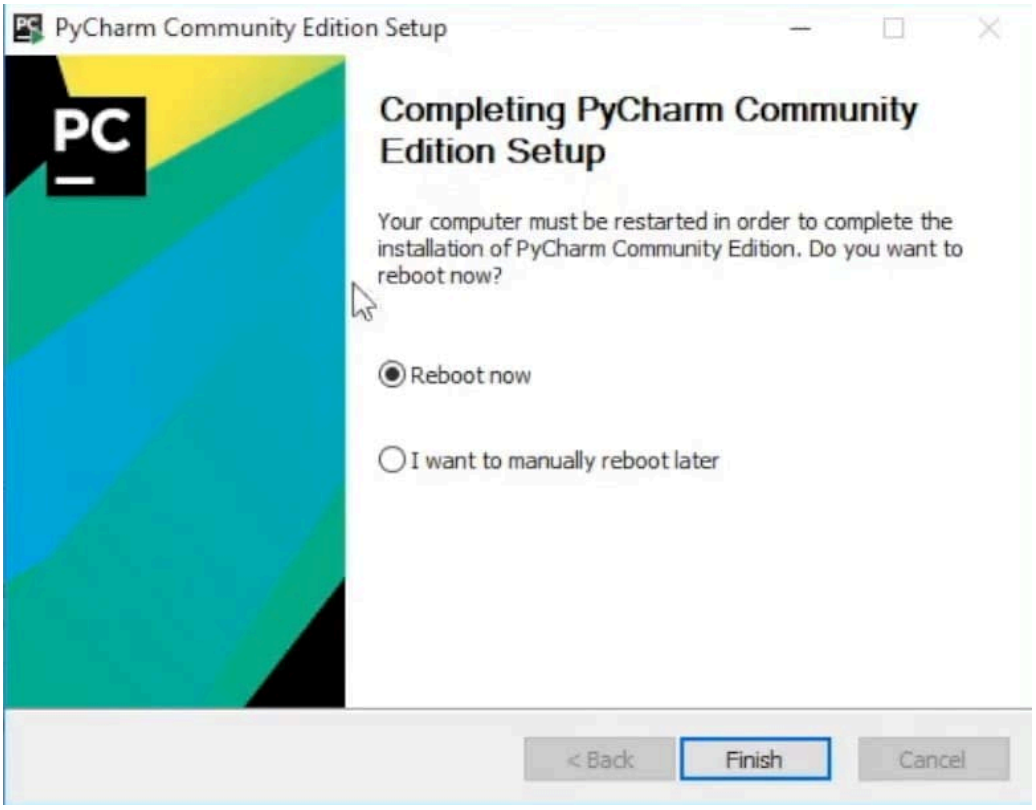
- เลือกตามในภาพ ยกเว้นว่าคอมพิวเตอร์ของใครที่เป็น 32 bit ก็เลือกเป็น 32-bit launcher แทน



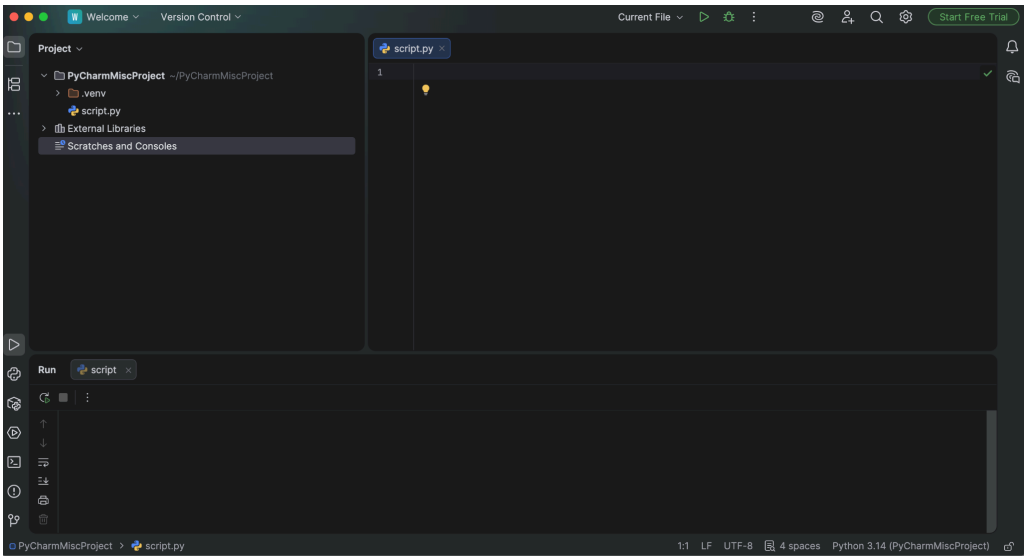
- ถัดไปกด Next และกด Install



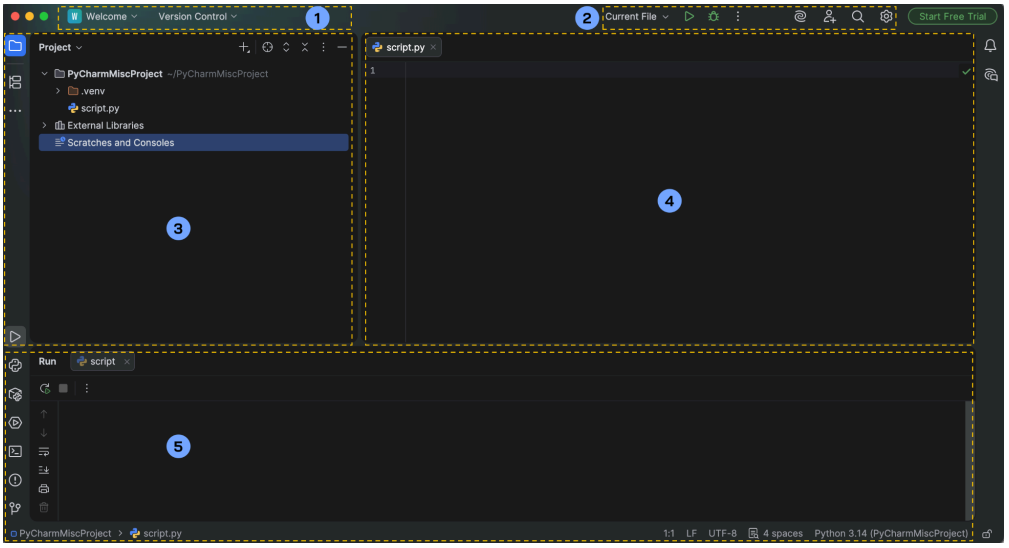
- เลือก Reboot now



2.3.2. โครงสร้างของ Pycharm



จะแยกเป็นองค์ประกอบดังนี้

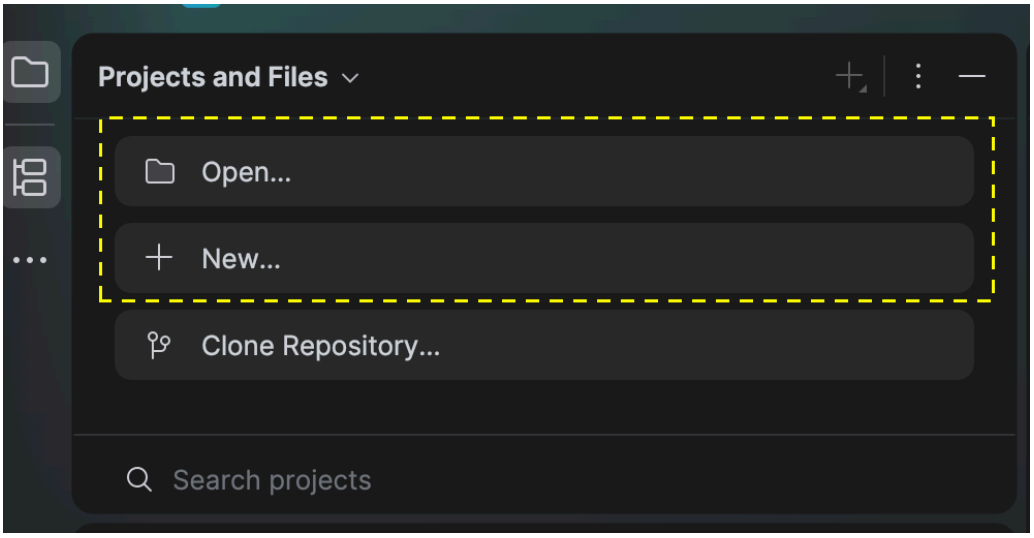


หมายเลข	บทบาท
①	ควบคุมภาพรวม & Version Control สามารถเปิดไฟล์ได้จากเมนูนี้
②	สั่งรัน / ดีบั๊ก
③	จัดการไฟล์ & Environment Project สามารถสร้างและเปิดโปรเจคได้ Structure สามารถดูลำดับของตัวแปรที่สร้างไว้บนพื้นที่เขียนโค้ดได้
④	พื้นที่เขียนโค้ด (Core Value)

หมายเลข	บทบาท
⑤	ดูผลลัพธ์ & แก้ปัญหา

2.3.3. การเริ่มต้นการใช้งาน Pycharm

การเปิดโฟลเดอร์การทำงาน/บันทึกการทำงาน



นักเรียน สามารถวสร้าง แผนงาน ใหม่ โดยที่เลือก บล็อคคำสั่ง New ระบบจะทำการสร้างไฟล์ไว้ให้และ เมื่อต้องการเรียกใช้ไฟล์เดิม สามารถกดไปยัง Open และทำงานต่อได้

คาบเรียนที่ 12: Program Flow และตัวแปร (60 นาที)

3. Program Flow และตัวแปร

3.1. Program Flow ของ Python

Program flow ของ Python คือ ลำดับการทำงานของคำสั่งต่างๆ ในโปรแกรม ซึ่งควบคุมโดยคำสั่งเงื่อนไข (if/else), ลูป (for/while) และการเรียกใช้ฟังก์ชัน เพื่อกำหนดว่าโค้ดส่วนไหนจะถูกรันเมื่อไหร่ จากเดิมที่รันจากบนลงล่างทีละบรรทัด

องค์ประกอบหลักที่ควบคุม Program Flow:

- **ลำดับ (Sequence):** การทำงานตามปกติจากบนลงล่างทีละบรรทัด (Default).
- **การเลือก (Selection/Conditional):** ใช้ if, elif, else เพื่อเลือกเส้นทางการทำงานตามเงื่อนไข (เช่น ตัดสินใจว่าลูกค้าจะได้ส่งฟรีหรือไม่).
- **การทำซ้ำ (Iteration/Looping):** ใช้ for, while เพื่อวนซ้ำการทำงานของโค้ด (เช่น ทำการคำนวณซ้ำๆ หรือประมวลผลข้อมูลทุกรายการ).
- **ฟังก์ชัน (Functions):** การเรียกใช้ฟังก์ชันเพื่อทำงานเฉพาะส่วน และควบคุมการไหลของโปรแกรมให้เข้า-ออกฟังก์ชัน.

3.2. Flowchart ของ Python

คือ แผนภาพที่ใช้สัญลักษณ์และลูกศรแสดงลำดับขั้นตอนการทำงาน (Algorithm) ของ

โปรแกรม Python เพื่อให้เข้าใจง่ายขึ้น ช่วยในการวางแผน, วิเคราะห์, แก้ไขโค้ด และสื่อสาร Logic ของโปรแกรมได้ชัดเจน โดยใช้รูปทรงมาตรฐาน เช่น วงรี (Start/End), สี่เหลี่ยมขนาน (Input/Output), สี่เหลี่ยม (Process) และเพชร (Decision/เงื่อนไข) เชื่อมโยมกันด้วยลูกศรบอกทิศทาง

1. Terminator: สัญลักษณ์เริ่มต้น และสิ้นสุด

สัญลักษณ์เริ่มต้น และสิ้นสุด ลักษะเป็นสี่เหลี่ยมผืนผ้าขอบมน หรือขอบโค้ง ใช้กำหนดจุดเริ่มต้น และจุดสิ้นสุดของกระบวนการทำงาน



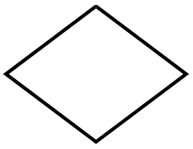
2. Process: สัญลักษณ์แสดงกระบวนการทำงาน

สัญลักษณ์แสดงกระบวนการทำงาน เป็นรูปสี่เหลี่ยมผืนผ้า ใช้เขียนแสดงขั้นตอนหรือกระบวนการทำงานโดยใช้ 1 สัญลักษณ์ต่อ 1 กระบวนการ เช่น ต้มน้ำ, เทน้ำใส่แก้ว ฯลฯ



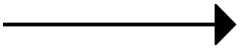
3. Decision: สัญลักษณ์แสดงเกณฑ์ของทางเลือกเพื่อการตัดสินใจ

สัญลักษณ์แสดงการตัดสินใจ ลักษณะเป็นรูปสี่เหลี่ยมข้าวหลามตัด ใช้งานโดยการเขียนเงื่อนไขการเปรียบเทียบ หรือเกณฑ์ในการเลือก



4. Connector: ลูกศรเชื่อมต่อขั้นตอนขั้นตอน

ลูกศรเชื่อมต่อขั้นตอน เป็นลูกศรสำหรับชี้บอกลำดับขั้นตอนในการทำงานจากขั้นตอนหนึ่ง ไปยังอีกขั้นตอนหนึ่ง โดยลากเส้นจากขั้นตอนก่อนหน้า และหัวลูกศรจะชี้อยู่ติดกับสัญลักษณ์ของขั้นตอนถัดไปเสมอ



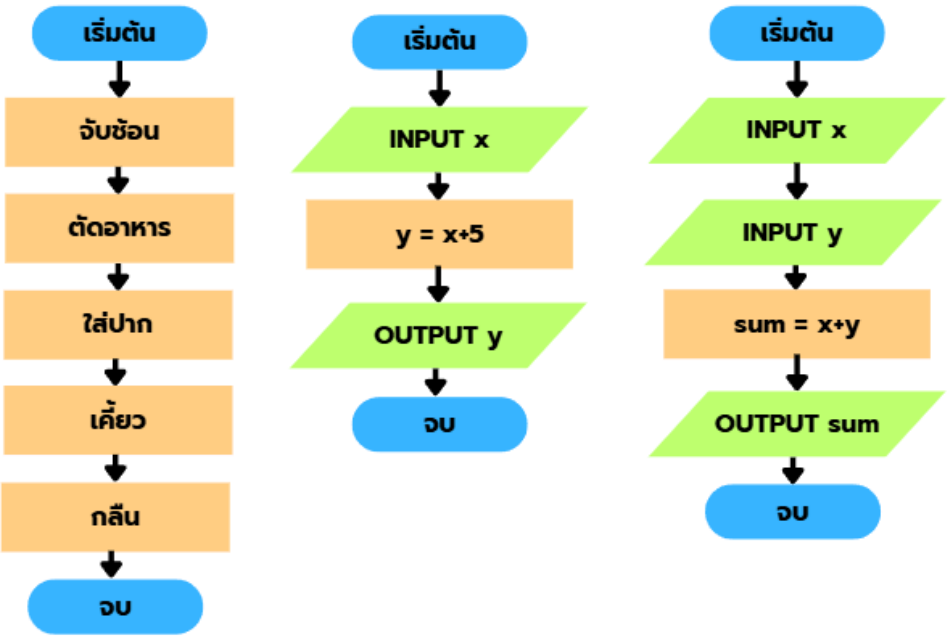
5. Input/Output: สัญลักษณ์แสดงข้อมูลเข้า/ออก

สัญลักษณ์แสดงข้อมูลเข้า/ออก ลักษณะเป็นรูปสี่เหลี่ยมคางหมู ใช้แสดงขั้ยตอนกระบวนการรับ หรือส่งข้อมูลออก เช่น ใช้กับกระบวนการที่คอมพิวเตอร์ต้องรับข้อมูลจากผู้ใช้งาน หรือให้คอมพิวเตอร์แสดงข้อมูลให้ผู้ใช้งานทราบ

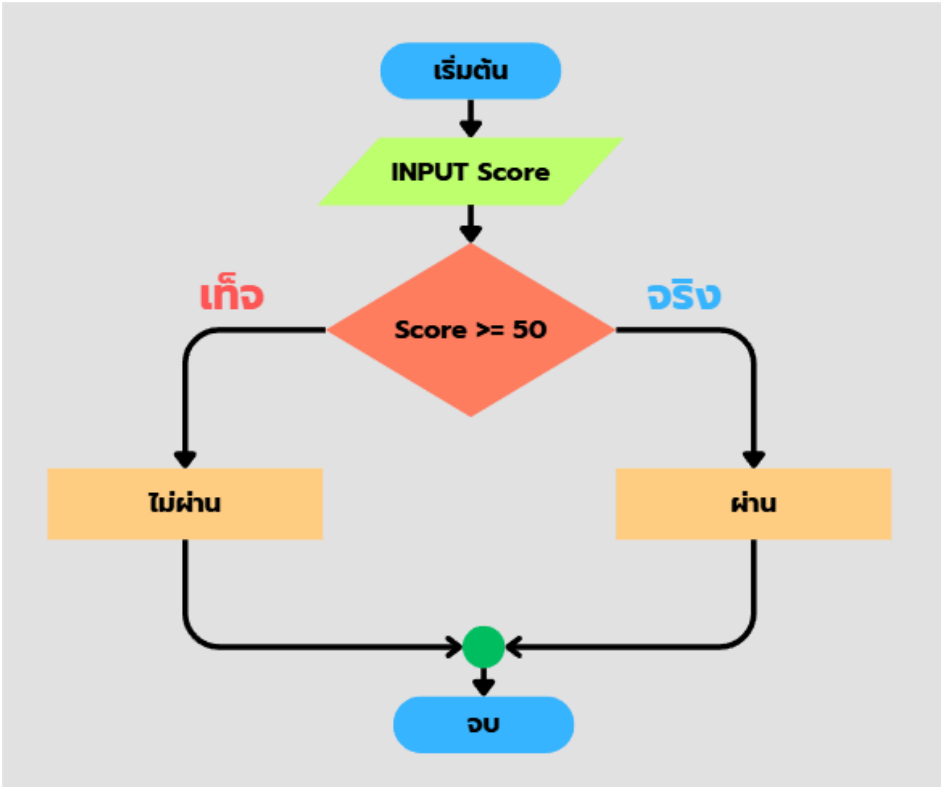


รูปแบบการทำงานของ Python ที่แสดงในผังงาน

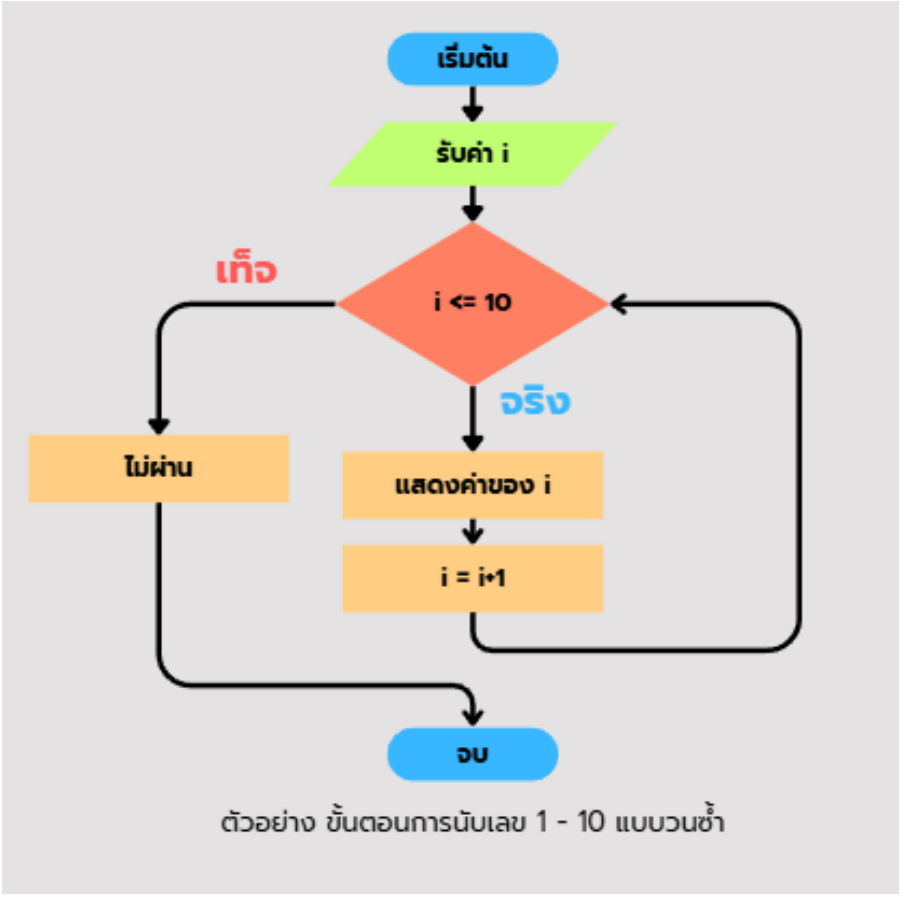
1. **ผังงานแบบเรียงลำดับ (Sequential Flow):** ทำงานทีละคำสั่งตามลำดับจากบนลงล่าง



2. **ผังงานแบบมีเงื่อนไข (Conditional Flow):** มีการตัดสินใจ (If/Else) ว่าจะไปทางไหนต่อตามเงื่อนไข

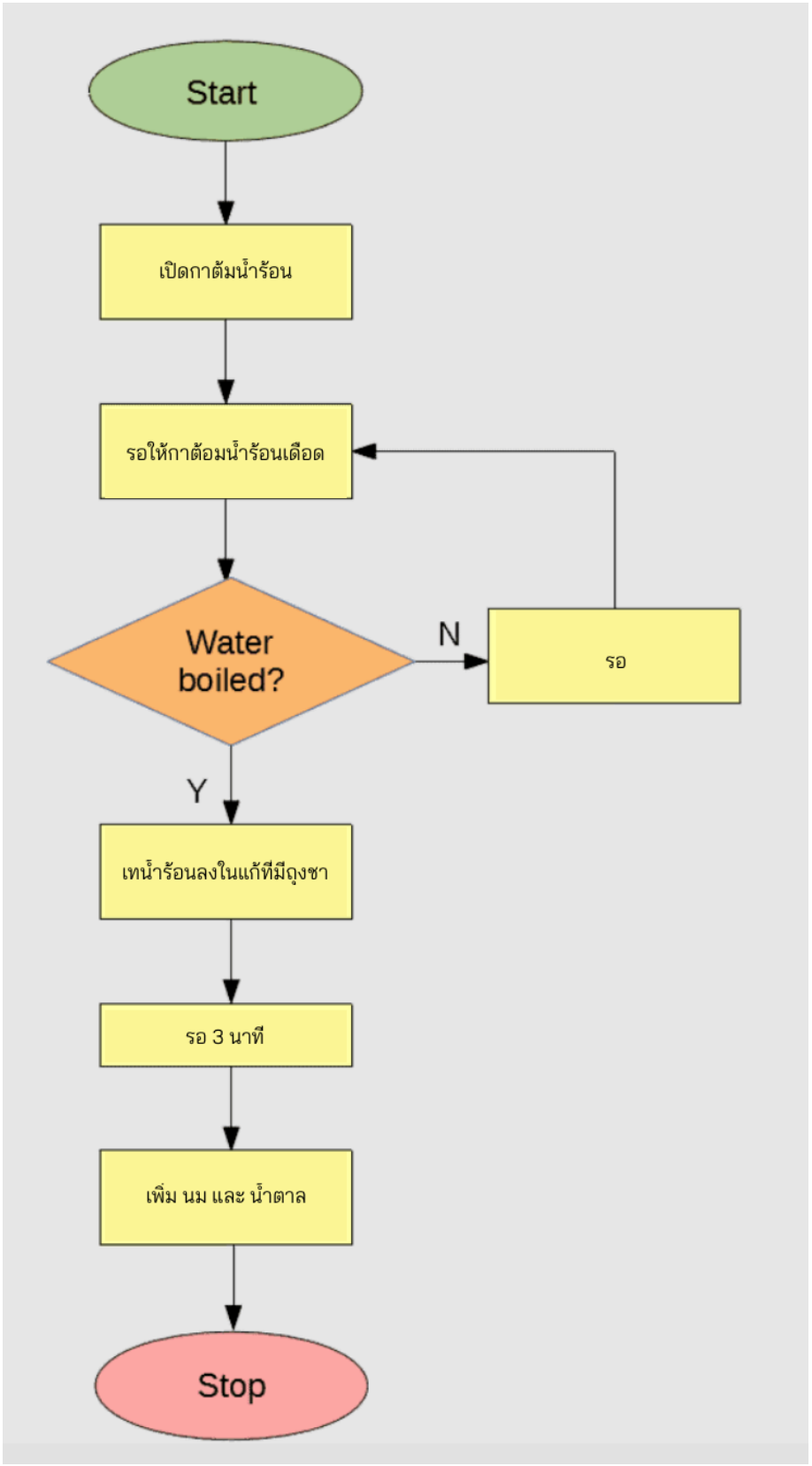


3. **ผังงานแบบทำซ้ำ (Looping Flow):** มีการทำงานซ้ำๆ (For/While Loop) จนกว่าจะครบเงื่อนไข



ตัวอย่าง ขั้นตอนการนับเลข 1 - 10 แบบวนซ้ำ

ตัวอย่างการทำงานของ Flowchart เพื่อให้มองเห็นภาพ ตรรกการทำงานของ Python



แนวคิด	สิ่งที่นักเรียนได้รับ
ลำดับขั้นตอน	โปรแกรมทำทีละขั้น
เงื่อนไข	ถ้าเดือด / ถ้ายังไม่เดือด
การวนซ้ำ	ยังไม่เดือดก็รอแล้วเช็คใหม่
ไม่กระโดดขั้น	ต้องเดือดก่อน ถึงจะเทน้ำ

Flowchart คือสิ่งที่บอกว่าโปรแกรมจะทำอะไรก่อน–หลัง Python คือการพิมพ์ตาม Flowchart ทีละขั้น ถ้า Flowchart ถูก โค้ดจะเขียนง่ายมาก

ข้อดีของการใช้ Flow chart คือ ช่วยให้เข้าใจขั้นตอนการทำงานได้ชัดเจนขึ้น วิเคราะห์และแก้ไขปัญหาได้ง่ายขึ้น สื่อสารและอธิบายระบบให้ผู้อื่นเข้าใจได้ดีขึ้น รวมถึงช่วยในการวางแผน ออกแบบ และปรับปรุงระบบให้มีประสิทธิภาพ

1. เข้าใจ logic ของโปรแกรมได้ง่ายขึ้น Flowchart ช่วยให้ visualize ลำดับขั้นตอน logic และอัลกอริทึมของโปรแกรม ช่วยให้นักพัฒนาเข้าใจภาพรวม วิเคราะห์ปัญหา และแก้ปัญหาได้ง่ายขึ้น ตัวอย่างเช่น สมมติว่ามีโปรแกรมสำหรับคำนวณค่าเฉลี่ยของตัวเลข Flowchart จะแสดงขั้นตอนต่างๆ ดังนี้

2. เขียนโปรแกรมได้อย่างมีประสิทธิภาพ Flowchart ช่วยให้นักพัฒนาเขียนโปรแกรมได้อย่างมีลำดับ ชัดเจน รัดกุม และสามารถติดตามตรวจสอบขั้นตอนได้ง่าย ช่วยลดข้อผิดพลาดที่อาจเกิดจากการเขียนโค้ดโดยไม่ได้วางแผน ช่วยให้นักพัฒนาเขียนโปรแกรมได้รวดเร็ว ประหยัดเวลาและทรัพยากร

3. สื่อสารกับผู้อื่นได้ง่ายขึ้น Flowchart ช่วยให้นักพัฒนาสามารถสื่อสาร อธิบาย และแลกเปลี่ยนความคิดเกี่ยวกับโปรแกรมกับผู้อื่นได้ง่ายขึ้น ช่วยให้ผู้ที่ไม่เข้าใจการเขียนโปรแกรม เข้าใจ logic และอัลกอริทึม ของโปรแกรมได้ง่ายขึ้น ช่วยให้การทำงานเป็นทีมมีประสิทธิภาพ

4. แก้ไขข้อผิดพลาดได้ง่ายขึ้น Flowchart ช่วยให้นักพัฒนาสามารถระบุจุดผิดพลาด วิเคราะห์สาเหตุ และแก้ไขข้อผิดพลาดในโปรแกรมได้ง่ายขึ้น ช่วยให้นักพัฒนาสามารถมองภาพรวมของโปรแกรม และระบุจุดที่อาจเกิดข้อผิดพลาดได้ง่าย ช่วยให้นักพัฒนาสามารถทดสอบโปรแกรมและแก้ไขข้อผิดพลาดได้อย่างมีประสิทธิภาพ

5. ประโยชน์อื่นๆ Flowchart ช่วยให้นักพัฒนาวิเคราะห์และออกแบบโปรแกรมก่อนการเขียนโค้ด ช่วยให้ประหยัดเวลา และทรัพยากร ช่วยให้นักพัฒนาเขียนโปรแกรมที่มี modularity และ reusable code ช่วยให้นักพัฒนาเขียนโปรแกรมที่เข้าใจง่าย

ชุดแบบฝึกหัดที่1 Flowchart (20 นาที)

โจทย์ 1: โปรแกรมทักทาย

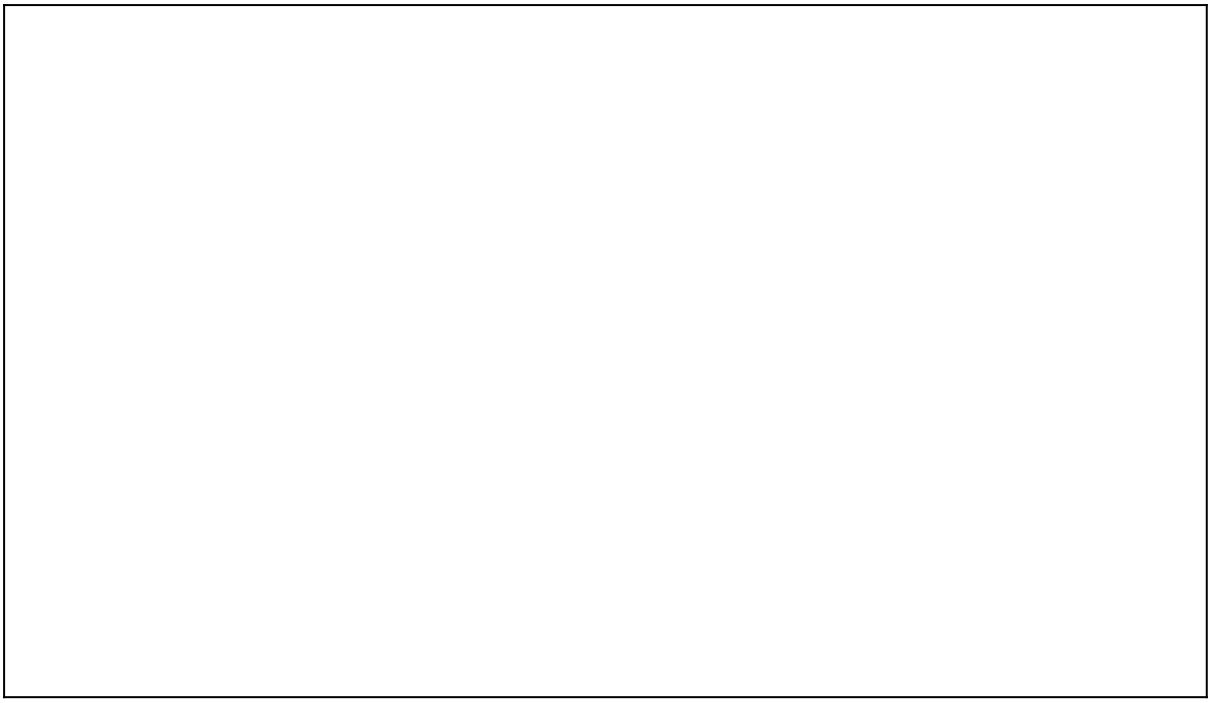
- รับชื่อจากผู้ใช้
- แสดงข้อความ “สวัสดี + ชื่อ”

โจทย์ 2: ตรวจสอบอายุ

- รับค่าอายุ
- ถ้าอายุ ≥ 18 → แสดง “คุณเป็นผู้ใหญ่”
- ถ้าน้อยกว่า → แสดง “คุณยังไม่บรรลุนิติภาวะ”

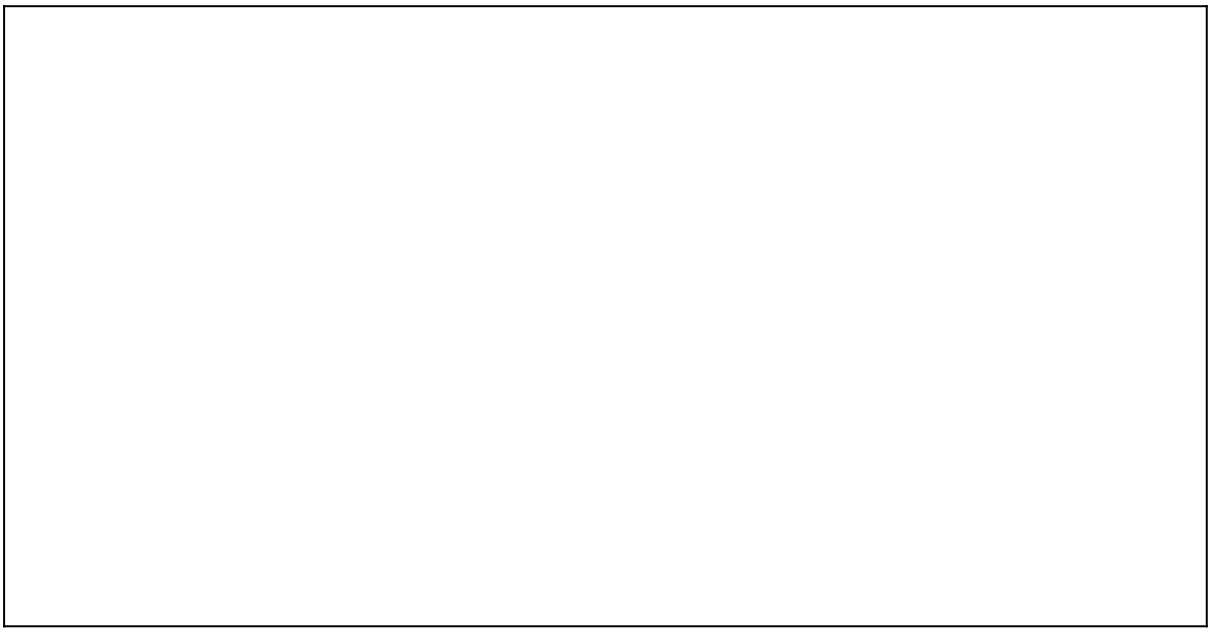
โจทย์ 3: ให้เกรด

- รับคะแนน (0–100)
- 80–100 → A
- 70–79 → B
- 60–69 → C
- ต่ำกว่า 60 → F



โจทย์ 4: นับเลข 1 ถึง 10

- แสดงตัวเลข 1–10 ทีละตัว



3.3. ตัวแปร (Variable)

หน้าที่: ใช้เก็บข้อมูลและนำไปใช้ต่อได้ภายหลัง.

3.3.1. การกำหนด: ชื่อตัวแปร = ค่า (เช่น age = 25, name = "Alex").

3.3.2. กฎการตั้งชื่อ

- ขึ้นต้นด้วยตัวอักษร (a-z, A-Z) หรือเครื่องหมาย _ (Underscore).
- ประกอบด้วยตัวอักษร, ตัวเลข (0-9), และ _
- ตัวพิมพ์เล็ก-ใหญ่ต่างกัน (Case-sensitive).
- ห้ามใช้คำสงวน (Reserved Words)

3.4. ชนิดข้อมูลหลัก (Key Data Types)

- **int (Integer):** จำนวนเต็ม (เช่น 10, -5).
- **float (Floating-Point Number):** จำนวนทศนิยม (เช่น 3.14, 100.50).
- **str (String):** ข้อความ (เช่น "Hello", 'Python').
- **list (List):** ข้อมูลหลายอย่างเก็บรวมกันในลำดับ (เช่น ["apple", "banana"]).
- **dict (Dictionary):** เก็บข้อมูลเป็นคู่ Key-Value (เช่น {"name": "John", "age": 30}).
- **bool (Boolean):** ค่าความจริง (True/False).

3.5. Operator (ตัวดำเนินการ): เครื่องหมายคำนวณ (+, -, *, /) และเปรียบเทียบ (==, <, >).

3.5.1. print(): คำสั่งแสดงผลลัพธ์ออกหน้าจอ.

3.5.2. input(): คำสั่งรับข้อมูลจากผู้ใช้.

3.6. Control Flow (การควบคุมการทำงาน): if, else, elif, for, while สำหรับการตัดสินใจและวนซ้ำ.

3.7. Function (ฟังก์ชัน): ชุดคำสั่งที่ทำงานเฉพาะอย่าง, สร้างด้วย def

3.8. Reserved Words

ในภาษา Python คำสงวน (หรือคำหลัก) คือคำที่กำหนดไว้ล่วงหน้าซึ่งมีความหมายและไวยากรณ์พิเศษในภาษา คำเหล่านี้ไม่สามารถใช้เป็นตัวระบุ (ชื่อสำหรับตัวแปร ฟังก์ชัน คลาส ฯลฯ) ได้เนื่องจากคำเหล่านี้กำหนดไวยากรณ์และโครงสร้างของโปรแกรม Python

Keywords in Python programming language				
False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

4. ความแตกต่างระหว่าง Scracth และ Python

มิติ	Scratch	Python
รูปแบบ	Block-based (ลากวาง)	Text-based (พิมพ์โค้ด)
ความยากเริ่มต้น	⭐ ง่ายมาก	⭐ ⭐ ยากกว่าเล็กน้อย
ภาระทางภาษา	ต่ำ (ไม่ต้องพิมพ์)	ปานกลาง (ต้องสะกดให้ถูก)
Bug	แทบไม่มี Syntax Error	มี Error จริง (เรียนรู้จากความผิดพลาด)
Mindset ที่ได้	ลำดับขั้นตอน	คิดเป็นระบบ + Debug
ความรู้สึกเด็ก	“เล่นเกม”	“ฉันเขียนโปรแกรมจริง”
ความจริงของโลกงาน	ไม่ใช้จริง	ใช้จริงในอุตสาหกรรม
ต่อยอด AI /Robotics	จำกัด	ต่อยอดได้ตรงสาย

คาบเรียนที่ 13: การเขียนโปรแกรม (60 นาที)

5. การเขียนโปรแกรม

5.1. ตัวอย่างการเขียนคำสั่งอย่างง่าย

เนื้อหาที่ 1: คำสั่ง print() - การแสดงผล (10 นาที)

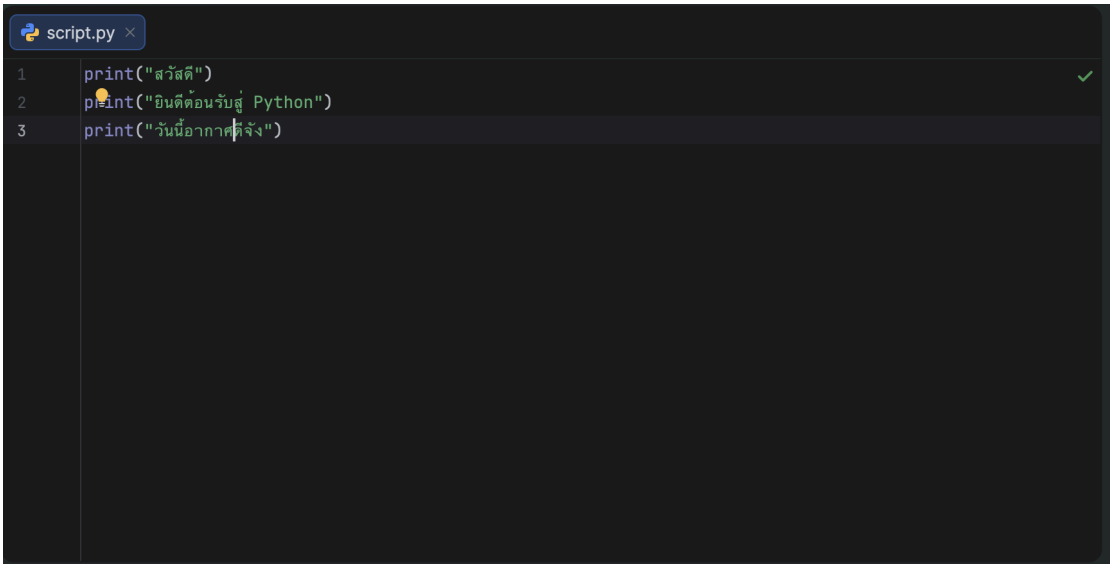
print() คืออะไร?

คำสั่ง "พูด" หรือ "แสดงผล" ให้ผู้ใช้เห็น เหมือนการพูดออกมา หรือเขียนบนกระดาน ใช้บ่อยที่สุดใน Python!

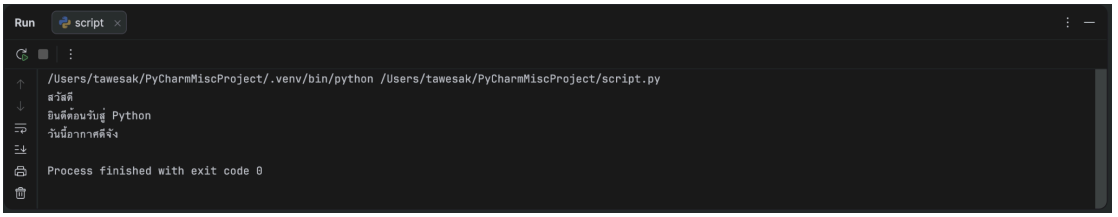
วิธีใช้:

```
print("ข้อความที่ต้องการแสดง")
```

ตัวอย่างที่ 1 - แสดงข้อความ:



ผลลัพธ์:



ตัวอย่างที่ 2 - แสดงผลการคำนวณ:

script.py

```python
print("5 + 3 =", 5 + 3)
print("10 - 2 =", 10 - 2)
print("4 * 5 =", 4 * 5)
print("20 / 4 =", 20 / 4)
```

ผลลัพธ์:

Run script

```
/Users/tawesak/PyCharmMiscProject/.venv/bin/python /Users/tawesak/PyCharmMiscProject/script.py
5 + 3 = 8
10 - 2 = 8
4 * 5 = 20
20 / 4 = 5.0

Process finished with exit code 0
```

ตัวอย่างที่ 3- ใช้กับตัวแปร:

script.py

```python
name = "สมหญิง"
age = 14
school = "โรงเรียนดี"
print("ข้อมูลนักเรียน")
print("ชื่อ:", name)
print("อายุ:", age)
print("โรงเรียน:", school)
```

ผลลัพธ์:

Run script

```
/Users/tawesak/PyCharmMiscProject/.venv/bin/python /Users/tawesak/PyCharmMiscProject/script.py
ข้อมูลนักเรียน
ชื่อ: สมหญิง
อายุ: 14
โรงเรียน: โรงเรียนดี

Process finished with exit code 0
```

เนื้อหาที่ 2: คำสั่ง input() - การรับข้อมูล

input() คืออะไร?

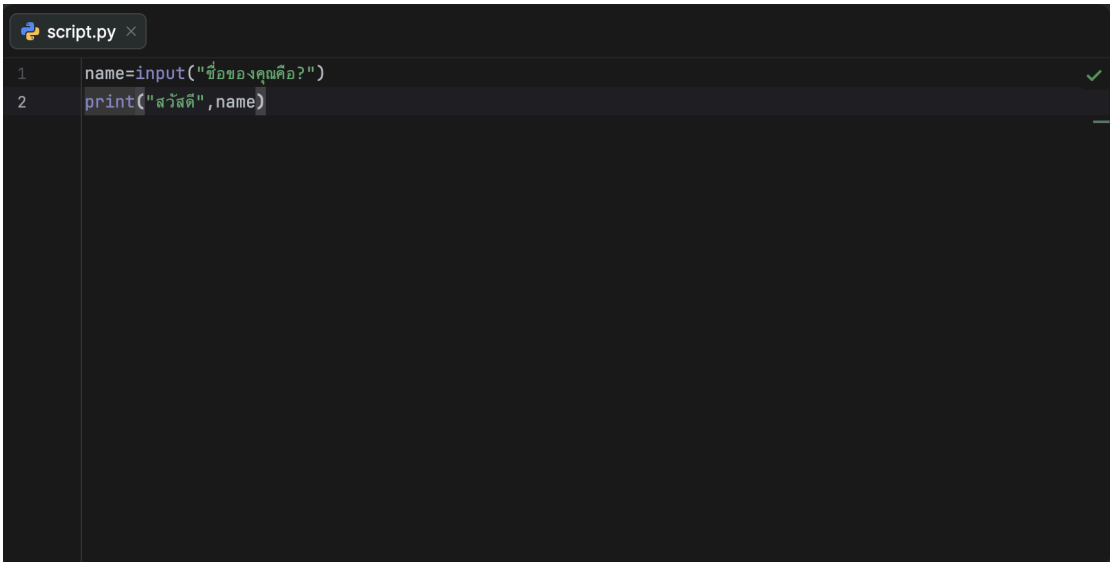
สิ่งที่ได้รับเข้ามา ซึ่งในการเขียนโปรแกรม Input จะหมายถึงการที่โปรแกรมนั้นจะรับค่าอะไรมาซักอย่าง เพื่อที่จะให้นำไปใช้ในขณะที่โปรแกรมกำลังทำงาน
โดยในภาษา Python นั้นจะใช้คำสั่ง input() เพื่อทำการรับ input เข้ามาสู่ระบบ

- คำสั่ง "ถามคำถาม" หรือ "รับข้อมูล" จากผู้ใช้
- เหมือนการถามคำถามแล้วรอคำตอบ
- ผลที่ได้จะเป็น String เสมอ!

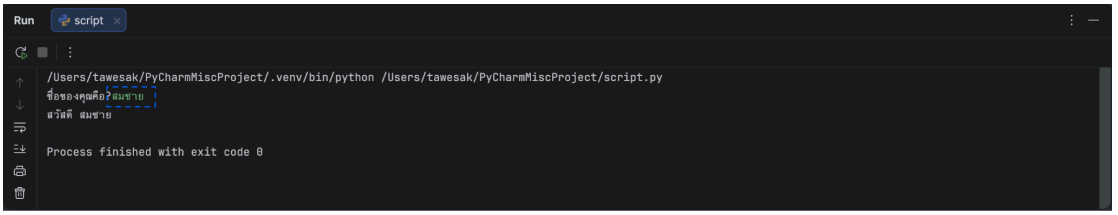
วิธีใช้:

```
ตัวแปร = input("คำถาม: ")
```

ตัวอย่างที่ 1 - ถามชื่อ:



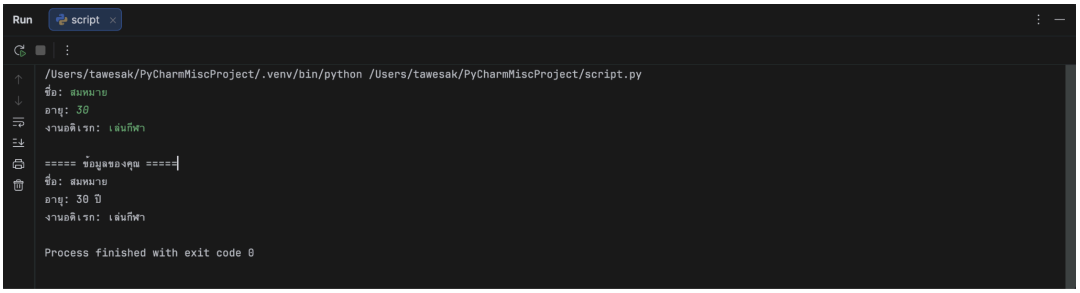
ผลลัพธ์:



ตัวอย่างที่ 2 - ถามข้อมูลหลายอย่าง:



ผลลัพธ์:



ข้อควรระวัง

```
age = input("อายุ: ") # ได้เป็น String
print(age + 5) # ❌ ERROR! ไม่สามารถบวกได้

# ต้องแปลงเป็น int ก่อน
age = int(input("อายุ: ")) # แปลงเป็น int
print(age + 5) # ✅ ได้!
```

ทำไมต้องแปลง?

- **input()** ให้ผลเป็น **String** เสมอ
- ถ้าต้องการนำมาคำนวณ → ต้องแปลงเป็นตัวเลข

ตัวอย่างคำสั่งแปลงชนิดข้อมูล:

1. **int()** - แปลงเป็นจำนวนเต็ม

```
age = input("อายุ: ")       # ได้ "13" (String)
age = int(age)              # แปลงเป็น 13 (int)
next_year = age + 1
print("ปีหน้าอายุ", next_year)
```

2. **float()** - แปลงเป็นทศนิยม

```
height = input("ส่วนสูง (ซม.): ")   # ได้ "155.5" (String)
height = float(height)              # แปลงเป็น 155.5 (float)
print("ส่วนสูง", height, "ซม.")
```

3. **str()** - แปลงเป็นข้อความ

```
score = 95                  # int
text = "คะแนนคือ " + str(score)
print(text)                 # คะแนนคือ 95
```

เทคนิค - แปลงทันทีตอน input:

```
# แทนที่จะทำ 2 บรรทัด
age = input("อายุ: ")
age = int(age)

# ทำเป็นบรรทัดเดียวได้!
age = int(input("อายุ: "))

# ตัวอย่างอื่น
height = float(input("ส่วนสูง: "))
price = float(input("ราคา: "))
```

5.2. เนื้อหาที่ 4: การคำนวณทางคณิตศาสตร์ (5 นาที)

ตัวดำเนินการคณิตศาสตร์ใน Python:

เครื่องหมาย	ความหมาย	ตัวอย่าง	ผลลัพธ์
+	บวก	10 + 5	15
-	ลบ	10 - 5	5
*	คูณ	10 * 5	50
/	หาร	10 / 2	5.0
**	ยกกำลัง	2 ** 3	8 (2×2×2)
//	หารปัดเศษลง	10 // 3	3
%	หารเอาเศษ	10 % 3	1

ตัวอย่างการใช้งาน:

การบวก ลบ คูณ หาร:

```
print("10 + 5 =", 10 + 5) # 15
print("10 - 5 =", 10 - 5) # 5
print("10 * 5 =", 10 * 5) # 50
print("10 / 2 =", 10 / 2) # 5.0
```

ยกกำลัง ():**

```
print("2 ยกกำลัง 3 =", 2 ** 3) # 8 (2×2×2)
print("5 ยกกำลัง 2 =", 5 ** 2) # 25 (5×5)
print("10 ยกกำลัง 3 =", 10 ** 3) # 1000
```

ใช้กับตัวแปร:

```
x = 10
y = 3

sum_result = x + y
minus_result = x - y
multiply_result = x * y
divide_result = x / y

print("ผลบวก:", sum_result) # 13
print("ผลลบ:", minus_result) # 7
```

```
print("ผลคูณ:", multiply_result) # 30
print("ผลหาร:", divide_result) # 3.333...
```

การคำนวณจริง:

```
# คำนวณพื้นที่สี่เหลี่ยม
width = 5
length = 10
area = width * length
print("พื้นที่ =", area, "ตร.ม.") # 50 ตร.ม.

# คำนวณค่าเฉลี่ย
score1 = 80
score2 = 90
score3 = 70
average = (score1 + score2 + score3) / 3
print("คะแนนเฉลี่ย =", average) # 80.0
```

กิจกรรมปิดท้าย

1. ให้นักเรียนเลือก 2-3 โปรแกรมมาลองเขียนตาม
2. ท้าทาย: ให้สร้างโปรแกรมของตัวเองตามความสนใจ เช่น:
    - โปรแกรมแปลงหน่วย (°C → °F)
    - โปรแกรมคำนวณอายุ (ปี → เดือน → วัน)
    - โปรแกรมคำนวณ BMI
    - โปรแกรมแนะนำตัวเองแบบสร้างสรรค์
3. นักเรียนที่ทำเสร็จเร็ว อาสานำเสนอหน้าชั้น

Post-test (แบบทดสอบหลังเรียน)

คำชี้แจง: ให้นักเรียนทำเครื่องหมาย ✓ ลงในช่องที่ถูกต้องที่สุด

ส่วนที่ 1: ความรู้พื้นฐาน (10 คะแนน)

1. Python ถูกสร้างโดยใคร?
    - ก. Mark Zuckerberg
    - ข. Guido van Rossum
    - ค. Bill Gates
    - ง. Steve Jobs
2. ข้อใดไม่ใช่ข้อดีของ Python?
    - ก. เรียนรู้ง่าย
    - ข. มี Library มากมาย
    - ค. ทำงานได้เร็วที่สุดในทุกภาษา
    - ง. ใช้ได้หลาย Platform
3. Scratch และ Python ต่างกันอย่างไร?
    - ก. Scratch ใช้บล็อก Python ใช้โค้ด
    - ข. Scratch ใช้โค้ด Python ใช้บล็อก
    - ค. ไม่ต่างกัน
    - ง. ใช้งานเหมือนกันทุกอย่าง
4. โปรแกรม Python ทำงานตามลำดับอย่างไร?
    - ก. บนลงล่าง
    - ข. ล่างขึ้นบน
    - ค. กลางออกข้าง
    - ง. สุ่มทำงาน
5. ข้อใดเป็นชื่อตัวแปรที่ถูกต้อง?
    - ก. 2students
    - ข. student-name
    - ค. student_name
    - ง. class
6. คำสั่งใดใช้รับข้อมูลจากผู้ใช้?
    - ก. print()
    - ข. input()
    - ค. get()
    - ง. read()
7. ข้อมูลชนิด int คืออะไร?
    - ก. ข้อความ
    - ข. ทศนิยม
    - ค. จำนวนเต็ม
    - ง. True/False
8. ผลลัพธ์ของ 10 // 3 คือเท่าไร?
    - ก. 3.33
    - ข. 3
    - ค. 1

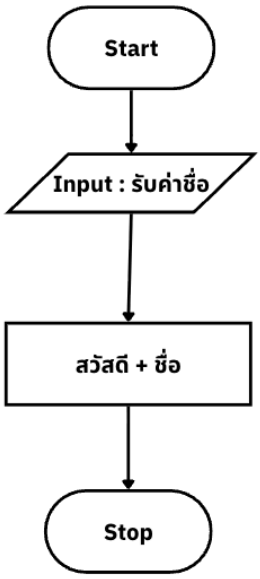
   - ง. 10
9. ฟังก์ชันใดใช้ตรวจสอบชนิดข้อมูล?
   - ก. check()
   - ข. type()
   - ค. data()
   - ง. typeof()
10. ข้อใดเป็น Reserved Word?
   - ก. variable
   - ข. data
   - ค. while
   - ง. python

เฉลย test:

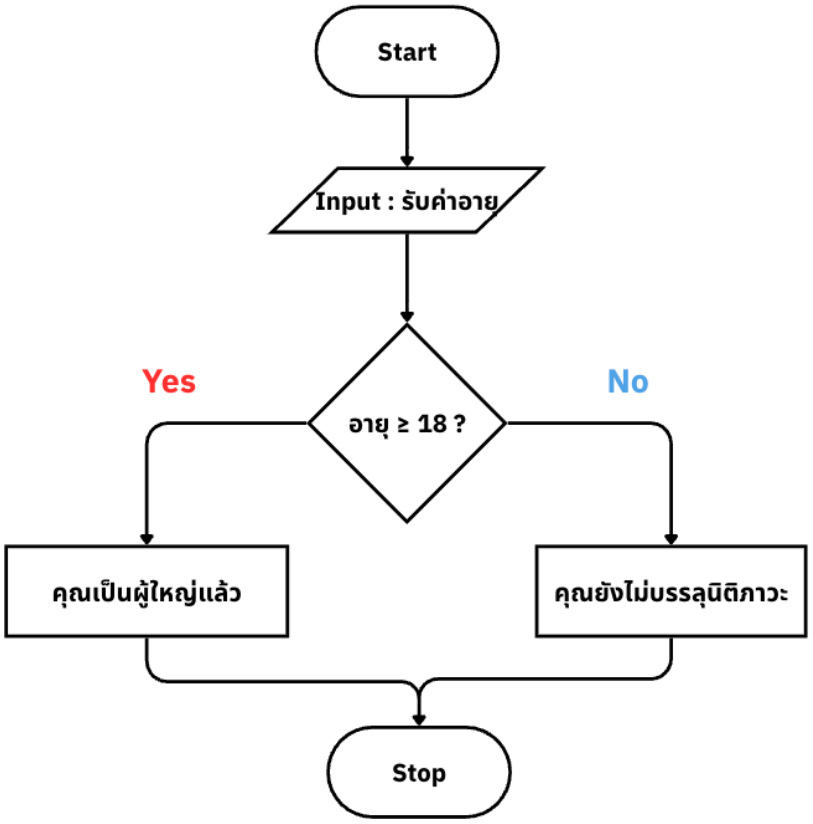
1.ข
2.ค
3.ก
4.ก
5.ค
6.ข
7.ค
8.ข
9.ข
10.ค

เฉลย ชุดแบบฝึกหัดที่ 1

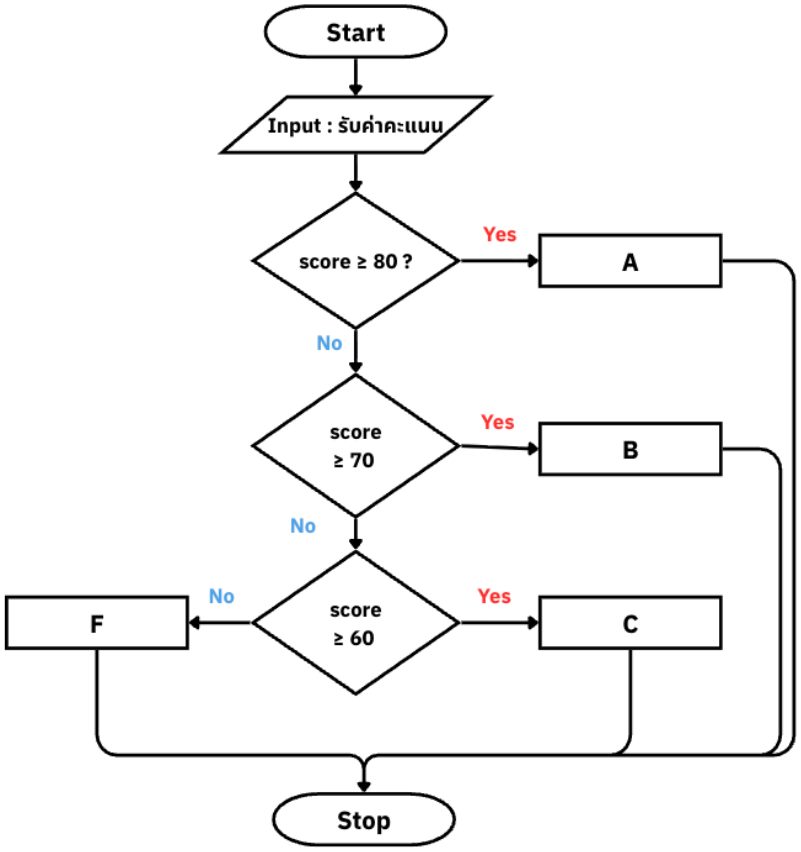
โจทย์ 1: โปรแกรมทักทาย



โจทย์ 2: ตรวจสอบอายุ



โจทย์ 3: ให้เกรด



โจทย์ 4: นับเลข 1 ถึง 10

